

Chapter 7 – Advanced SQL

Part 4: Views and Access Control

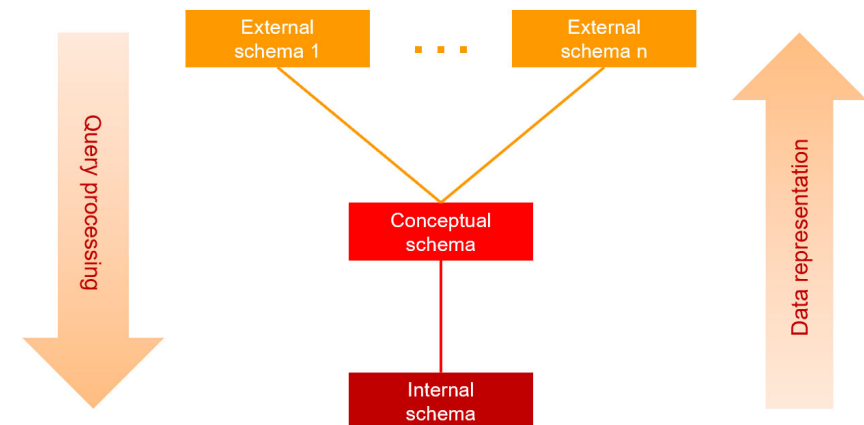
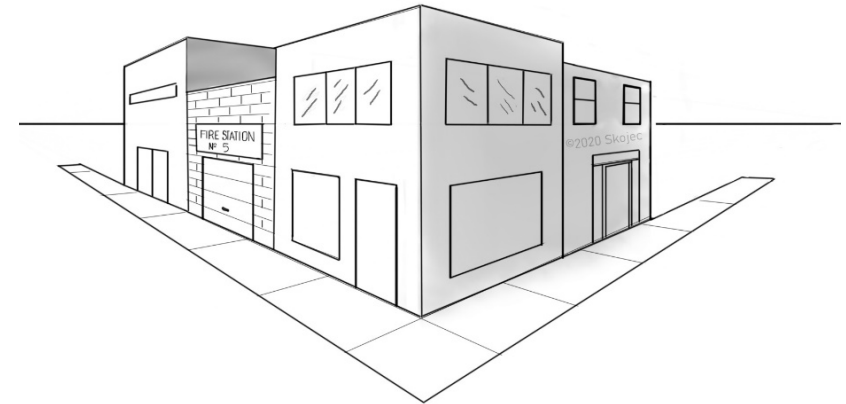
Databases lectures

Dr Kai Höfig



Views

- ◆ **Views** are virtual relations generated from base-tables (or other views, since a view is a relation itself) that show a certain aspect or part of the data.
- ◆ Views are external DB schemas following the 3-level schema architecture.
- ◆ Views replace complex queries and offer a simple to use interface for clients.
- ◆ Views are defined by queries.
- ◆ Using trigger and rights management, views can be used as a fully functional external schema interface for clients (logical data independence!).





Example of a simple view

- ◆ A query

```
select  staffs.first_name, staffs.last_name,
        manager.first_name+' '+manager.last_name as manager,
        stores.store_name
from    staffs
        join staffs as manager on(staffs.manager_id=manager.staff_id)
        join stores on (staffs.store_id=stores.store_id)
```

- ◆ ..can be stored as a view

```
create view v_staffs as
select staffs.first_name, staffs.last_name, manager.first_name+' '+m
from   staffs join staffs as manager on(staffs.manager_id=manager.st
```

- ◆ ..and now the client query is much easier.

```
select * from v_staffs;
```

- ◆ In the background this happens:

```
select * from (select staffs.first_name, staffs.last_name, manager.f
                from   staffs join staffs as manager on(staffs.manage
                ) as v_staffs
```



Problem areas with views

- ♦ Reading from views is **unproblematic**. Insert, update and delete operations are **problematic**, but required for a functioning user interface and a sound logical data independence.

- ♦ Example:

```
insert into v_staffs (first_name, last_name, manager, store_name)
values ('Lisa', 'Smith', 'Sarah Miller', 'New York');
```

Email
not null?

```
CREATE TABLE staffs (
  staff_id INT IDENTITY (1, 1) PRIMARY KEY,
  first_name VARCHAR (50) NOT NULL,
  last_name VARCHAR (50) NOT NULL,
  email VARCHAR (255) NOT NULL UNIQUE,
  phone VARCHAR (25),
  active tinyint NOT NULL,
  store_id INT NOT NULL,
  manager_id INT,
  FOREIGN KEY (store_id) REFERENCES stores (store_id) ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (manager_id) REFERENCES staffs (staff_id) ON DELETE NO ACTION ON UPDATE NO ACTION
);
```

ID?

```
CREATE TABLE stores (
  store_id INT IDENTITY (1, 1) PRIMARY KEY,
  store_name VARCHAR (255) NOT NULL,
  phone VARCHAR (25),
  email VARCHAR (255),
  street VARCHAR (255),
  city VARCHAR (255),
  state VARCHAR (10),
  zip_code VARCHAR (5)
);
```

ID? What if the
store does not
exist?



Projection views

- Views that only address one relation not using aggregate functions are called **projection views**.

```
create view v_Storelist as
select  store_name
from    stores;
```

- Changes to projection views are possible, when the values not in the projection list can be null and deleting or updating is not causing inconsistencies.

```
insert into v_Storelist
values ('New Bike Store');
```

is just fine, since
the other values
can be null.

```
CREATE TABLE stores (
  store_id INT IDENTITY (1, 1) PRIMARY KEY,
  store_name VARCHAR (255) NOT NULL,
  phone VARCHAR (25),
  email VARCHAR (255),
  street VARCHAR (255),
  city VARCHAR (255),
  state VARCHAR (10),
  zip_code VARCHAR (5)
);
```



Effect conformance for insert/update/delete in views

◆ Effect conformance

- User shall see the effect as if the update had been executed directly on the viewing relation.

store_name

Santa Cruz Bikes
Baldwin Bikes
Rowlett Bikes

```
insert into v_Storelist  
values ('New Bike Store');
```

Effect on view



store_name

Santa Cruz Bikes
Baldwin Bikes
Rowlett Bikes
New Bike Store

store_id store_name

1 Santa Cruz Bikes
2 Baldwin Bikes
3 Rowlett Bikes

Insert on base table



store_id store_name

1 Santa Cruz Bikes
2 Baldwin Bikes
3 Rowlett Bikes
5 New Bike Store



Violating effect conformance: tuple migration

- ◆ Tuple migration happens, when the inserted tuple does not appear in the view it was inserted to. Tuple migration violates effect conformity.

```
create view v_productlist2018 as
select    product_name,
          brand_id,
          category_id,
          model_year,
          list_price
from      products
where     model_year='2018'
```

```
select count(*)
from v_productlist2018;
```

205

```
insert into v_productlist2018 values
('My superBike',1,1,'2019',2000.99);
```

Effect on view



```
select count(*)
from v_productlist2018;
```

205  should be 206

- ◆ The tuple with 'My superBike' migrated out of the view. User cannot see the effect of his interactions in the view.



with check option to improve effect conformity

- ◆ Adding the command `with check option` to a view, monitors for every insert/update/delete operation whether the view is affected by evaluation the where clause defined for the view.

```
create view v_productlist2018 as
select  product_name,
        brand_id,
        category_id,
        model_year,
        list_price
from    products
where   model_year='2018'
with check option
```

```
insert into v_productlist2018
values ('My superBike',1,1,'2019',2000.99);
```

Clashes with where clause. Would migrate tuple outside of view. Insert possible without `with check option`!

The attempted insert or update failed because one or more rows resulting from the operation did not qualify under the `CHECK OPTION` constraint.



Join views

- ◆ Views that contain a join but no aggregate functions are called **join views**. Insert/update/delete operations on join views can not be translated automatically by the DBMS.
- ◆ Trigger provide a solution to specifically enable operations on join views depending on the application domain.

```
create trigger insert_into_v_something on v_something  
instead of insert as  
...
```



part of exercise



Minimality for insert/update/delete in views

◆ Minimality

- Base database should only be modified minimally in order to achieve the effect mentioned.
- Example: creating a backup of the entire table products when inserting a new product within a view would violate the minimality property.



Consistency preservation for insert/update/delete in views

◆ Consistency preservation

- Updating a view must not lead to integrity violations in the base database
- Example: adding an email address is required for every employee. Inserting a new employee in staffs without an email-address violates the not null constraint and is not allowed.



Data protection for insert/update/delete in views

◆ Respect for data protection

- If the view is introduced for data protection reasons, the consciously hidden part of the base database must not be affected by updates to the view.
- Example: a view that shows the products of one store cannot be used to add products for a different store.



Aggregation views

- ◆ Views that contain an aggregate function are called **aggregation views**. Insert/update/delete operations on aggregation views can not be translated automatically by the DBMS and also the manual translation using triggers makes no sense.

```
create view total_of_all_store_trunovers as
select sum(list_price*quantity*(1-discount)), store_id
from order_items join orders on (order_items.order_id=orders.order_id)
group by store_id
```

(Kein Spaltenname)	store_id
867542.2436	3
1605823.0365	1
5215751.2775	2



Summary of the problem areas

- ◆ Violation of the schema definition (e.g. insertion of null values for projection views)
- ◆ Data protection: avoid side effects on the non-visible part of the database (tuple migration, selection views)
- ◆ Not always unique transformation: selection problem
- ◆ Aggregation views (among others): no meaningful transformation possible
- ◆ Elementary view change should correspond with precisely one atomic change to base relation: 1:1 relationship between the view tuples and the tuples of the base relation (no projecting of keys)

Zugriffskontrolle



Rights assignment in database systems



- ◆ Who? User ID or user group
- ◆ Using? An operation like read, insert, update, delete, ...
- ◆ What? relations or views

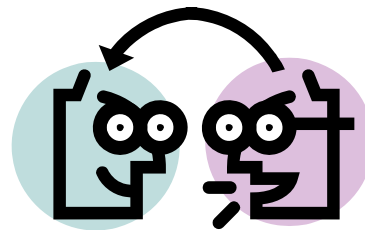


Rights assignment in SQL

```
grant    <Rights>
on       <Table>
to       <UserList>
        [with grant option]
```

◆ Explanations:

- In <Rights> list from select, insert, update, delete. All also possible, but deprecated since it also implies rights such as create database.
- After on: relation or view name
- After to: authorisation identifiers (also public, group)
- Special right: right to pass on rights (with grant option)





Withdrawal (revocation) of rights

```
revoke <Rights>  
on    <Table>  
from  <UserList>  
      [restrict | cascade ]
```

◆ Explanations:

- `restrict`: if rights have already been passed on to third parties: termination of `revoke`
- `cascade`: withdrawal of rights propagated using `revoke` to all users who received it from this user via `grant`



Role model from SQL:2003

- ◆ Roles were introduced from SQL:2003 to simplify the administration of rights
- ◆ Instead of giving rights directly to users, rights are assigned to roles and then roles to users → easier to transfer when a person changes
- ◆ Example

```
create view v_total_of_all_store_trunovers as
    select sum(list_price*quantity*(1-discount)) as turnover, store_id
    from order_items join orders on (order_items.order_id=orders.order_id)
    group by store_id;
```

```
create role store_managers;
```

```
grant select on v_total_of_all_store_trunovers to store_managers;
```



Example

```
create view v_all_products as
  select product_name, list_price
  from   products
```

- ◆ Authorisation for public: “Everyone can view the list prices of our products”

```
grant select on dbo.v_all_products to public
```

- ◆ Deauthorization for public: “No user that is only public can see the products base-table”

```
revoke select,insert,update,delete on products from public
```



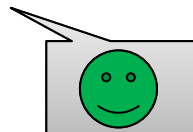
Example for a *public* interface

- ◆ Every SQL Server login belongs to the public server **role**. When a server principal hasn't been granted or denied specific permissions on a securable object, the user inherits the permissions granted on that object to the public role. So, to connect using the public role, we need a user login.

```
create login public_web_user with password = '';  
create user public_web_user for login public_web_user;
```

- ◆ Connecting to the db using this credentials enables to see the previously created view, but nothing more.

```
use BikeStore;  
select * from products;  
select * from v_all_products
```





Rights assignment in commercial DBMS

- ◆ Rights assignment in commercial DBMS is considerably more complex
- ◆ Usually, the concepts of **users and roles** remain the basis
- ◆ Often integration with other rights systems
 - Users of the operating system (e.g. Windows users)
 - Integration with identity management systems (e.g. Active Directory)
 - Own user administration of the DBMS
 - ➔ Often multiple systems like this are used simultaneously/in parallel
- ◆ Example
 - You log into your exercise db using a username/password combination when using docker and your windows credentials when the db is installed locally.